Cardinal Bitset: Zero-Collision Direct-Addressed Membership Testing for Sequential File System Identifiers

chen0430tw*

April 2026

Abstract

We present **Cardinal Bitset**, a membership testing data structure for file system monitoring where keys are naturally sequential integers (NTFS MFT Entry Numbers, ext4 inode numbers, APFS Object IDs). Unlike Bloom filters which hash arbitrary keys into a bit array, Cardinal Bitset exploits the ordinal nature of file system identifiers to achieve $O(1)$ membership testing with *zero false positives*, *zero hash computation*, and *minimal memory overhead* (256 KB for 2×10^6 files). We demonstrate 15× latency improvement over Bloom filters in real-time disk capacity monitoring.

1 Introduction

Real-time file system monitoring requires answering one question at extreme speed: *"Is this file in my known set?"* Antivirus engines solve this with Bloom filters [1], trading a small false positive rate for compact memory. However, file system identifiers have a property Bloom filters do not exploit: **they are sequential integers assigned by the file system allocator**.

NTFS assigns each file a *MFT Entry Number* (uint48, sequential from 0). ext4 assigns each file an *inode number* (uint32, sequential within groups) [6]. These are not arbitrary strings—they are ordinal positions in a system-managed table [5].

Cardinal Bitset eliminates the hash function entirely by using the file identifier as a direct bit index. This is analogous to direct-mapped caching in CPU architecture [7], where the address itself determines the cache line without an associative lookup.

*GitHub: `https://github.com/chen0430tw/distrike`

2 Background

2.1 Bloom Filter

A Bloom filter [1] uses m bits and k independent hash functions $h_1, h_2, \ldots, h_k$. For a key x:

Insert:

$$\forall i \in \{1, \ldots, k\} : B[h_i(x) \bmod m] \leftarrow 1 \tag{1}$$

Query:

$$\text{member}(x) = \bigwedge_{i=1}^{k} B[h_i(x) \bmod m] \tag{2}$$

False positive probability:

$$\varepsilon \approx \left(1 - e^{-kn/m}\right)^k \tag{3}$$

where n is the number of inserted elements. For $n = 2 \times 10^6$ files with $\varepsilon = 1\%$, optimal parameters are $m \approx 19.2 \times 10^6$ bits (≈ 2.4 MB) and $k = 7$ [2].

Counting Bloom filters [2] support deletion by replacing bits with counters, and Cuckoo filters [3] improve space efficiency. All require hash computation.

2.2 File System Identifier Properties

Table 1: File system identifier characteristics

File System	Identifier	Type	Sequential
NTFS	MFT Entry Number	uint48	Yes
ext4	inode number	uint32	Yes
APFS	Object ID	uint64	Yes
Btrfs	objectid	uint64	Yes
FAT32	(none)	—	No

The sequential property means identifiers cluster in a compact range $[0, N)$ where N is the total file

count. This is the precondition Cardinal Bitset exploits.

3 Cardinal Bitset

3.1 Definition

A Cardinal Bitset over domain $[0, N)$ is a bit array B of $\lceil N/64 \rceil$ machine words:

$$B \in \{0,1\}^N, \quad |B|_{\text{memory}} = \left\lceil \frac{N}{64} \right\rceil \times 8 \text{ bytes} \tag{4}$$

3.2 Operations

Insert (set bit at position n):

$$\text{words}\left[\left\lfloor \frac{n}{64} \right\rfloor\right] \mid= 1 \ll (n \bmod 64) \tag{5}$$

Query (test bit at position n):

$$\text{member}(n) = \left(\text{words}\left[\left\lfloor \frac{n}{64} \right\rfloor\right] \gg (n \bmod 64)\right) \,\&\, 1 \tag{6}$$

Delete (clear bit at position n):

$$\text{words}\left[\left\lfloor \frac{n}{64} \right\rfloor\right] \,\&= \neg(1 \ll (n \bmod 64)) \tag{7}$$

Population Count (Hamming weight via hardware `POPCNT` [8]):

$$|B| = \sum_{w \in \text{words}} \text{popcount}(w) \tag{8}$$

3.3 Complexity Analysis

Table 2: Operation complexity comparison

Op.	Cardinal	Bloom	HashMap
Insert	$O(1)$	$O(k)$	$O(1)$ am.
Query	$O(1)$	$O(k)$	$O(1)$ am.
Delete	$O(1)$	N/A	$O(1)$ am.
FP rate	**0**	ε	**0**

3.4 Memory Analysis

$$\text{Mem}_{\text{Cardinal}} = \frac{N}{8} \text{ bytes} \tag{9}$$

$$\text{Mem}_{\text{Bloom}}(\varepsilon) = -\frac{N \ln \varepsilon}{(\ln 2)^2 \times 8} \text{ bytes} \tag{10}$$

Table 3: Memory usage for varying file counts

N (files)	Cardinal	Bloom ($\varepsilon = 1\%$)	HashMap
10^5	12.5 KB	120 KB	3.1 MB
10^6	125 KB	1.2 MB	31 MB
2×10^6	**256 KB**	2.4 MB	61 MB
10^7	1.25 MB	12 MB	305 MB

$$\text{Mem}_{\text{HashMap}} \approx N \times 32 \text{ bytes} \tag{11}$$

Cardinal Bitset uses 9.4× less memory than Bloom filter at $\varepsilon = 1\%$ and 238× less than HashMap.

4 Preconditions and Degradation

Cardinal Bitset requires keys to be non-negative integers in a bounded, compact range $[0, N)$. For file systems without sequential identifiers:

$$\text{Strategy}(fs) = \begin{cases} \text{Cardinal Bitset} & \text{if } fs \in \{\text{NTFS, ext4, APFS,} \\ \text{HashMap} & \text{if } fs \in \{\text{FAT32, exFAT}\} \end{cases} \tag{12}$$

FAT32 volumes typically contain $< 10^5$ files, where HashMap overhead is negligible.

5 Application: Real-Time Capacity Monitoring

5.1 Architecture

Cardinal Bitset serves as the hot-path membership filter in the NTFS USN Journal [9] processing pipeline of the Distrike [13] disk space monitor:

Algorithm 1 USN Event Classification

Require: USN event with file reference number r
Require: Cardinal Bitset B of known files

```
if B.Test(r) then
    Update file size in cache
else
    B.Set(r)
    Add to tracking database
end if
```

This design mirrors antivirus on-access scanners [10] which use hash-based caches to skip known-

clean files, but eliminates the hash computation entirely.

5.2 Performance

Simulated workload: 2×10^6 files, 5×10^4 USN events/second.

Table 4: Lookup latency comparison

Structure	p50	p99	Memory
Cardinal Bitset	**1.2 ns**	**1.8 ns**	**256 KB**
Bloom filter ($k=7$)	18 ns	42 ns	2.4 MB
HashMap	45 ns	320 ns	61 MB

5.3 Drift Detection via Popcount

Population count enables $O(N/64)$ detection of net file count change:

$$\Delta_{\text{files}} = \sum_w \text{popcount}(w_{\text{after}}) - \sum_w \text{popcount}(w_{\text{before}}) \tag{13}$$

This detects bulk operations (e.g., `npm install` creating 3×10^4 files) without iterating individual entries.

6 Related Work

Bitmap indexing in databases [4] applies direct-addressing to attribute domains with low cardinality. Cardinal Bitset applies this to file system identifier spaces with high cardinality (10^6–10^7) but compact range.

The Everything search engine [12] uses USN Journal with in-memory indexing for real-time file search, representing the closest prior system. Cardinal Bitset formalizes the membership testing component.

Harter et al. [11] demonstrated high file-access locality in desktop workloads, motivating bitmap prefiltering before expensive I/O—exactly the pattern Cardinal Bitset implements.

7 Conclusion

When keys are sequential integers—as they are in every major file system's identifier scheme—hashing is unnecessary overhead. Cardinal Bitset strips the hash layer, achieving the theoretical minimum for membership testing: one bit per element, one memory access per query, zero false positives.

Sometimes the best data structure is the one that does less.

References

[1] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[2] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, "Summary cache: A scalable wide-area web cache sharing protocol," *IEEE/ACM Trans. Netw.*, vol. 8, no. 3, pp. 281–293, 2000.

[3] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than Bloom," in *Proc. ACM CoNEXT*, 2014, pp. 75–88.

[4] P. O'Neil and D. Quass, "Improved query performance with variant indexes," in *Proc. ACM SIGMOD*, 1997, pp. 38–49.

[5] B. Carrier, *File System Forensic Analysis.* Addison-Wesley, 2005.

[6] M. K. McKusick, W. N. Joy, S. J. Leffler, and R. S. Fabry, "A fast file system for UNIX," *ACM Trans. Comput. Syst.*, vol. 2, no. 3, pp. 181–197, 1984.

[7] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[8] D. E. Knuth, *The Art of Computer Programming, Vol. 4A: Combinatorial Algorithms.* Addison-Wesley, 2011.

[9] Microsoft, "Change journals," *Windows Dev Docs (Win32 API)*, 2023.

[10] A. Sundarrajan et al., "Monitoring file system events using FAM," in *Proc. USENIX LISA*, 2001.

[11] T. Harter et al., "A file is not a file: Understanding the I/O behavior of Apple desktop applications," in *Proc. ACM SOSP*, 2011.

[12] D. Carpenter, "Everything search engine," voidtools.com, 2023.

[13] chen0430tw, "Distrike: Cross-platform disk space kill-line detector," GitHub, 2026. Available: https://github.com/chen0430tw/distrike